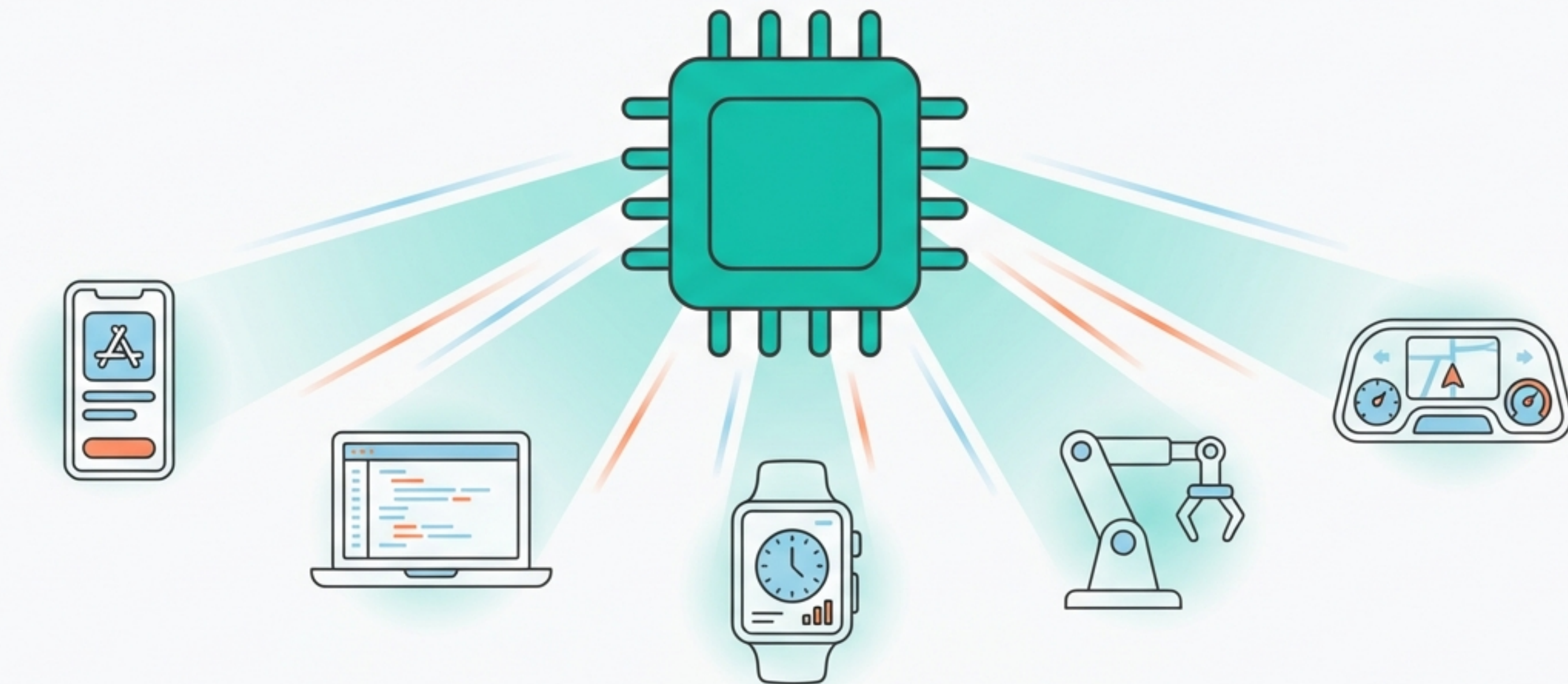


# What's New? AIエージェント開発の新時代

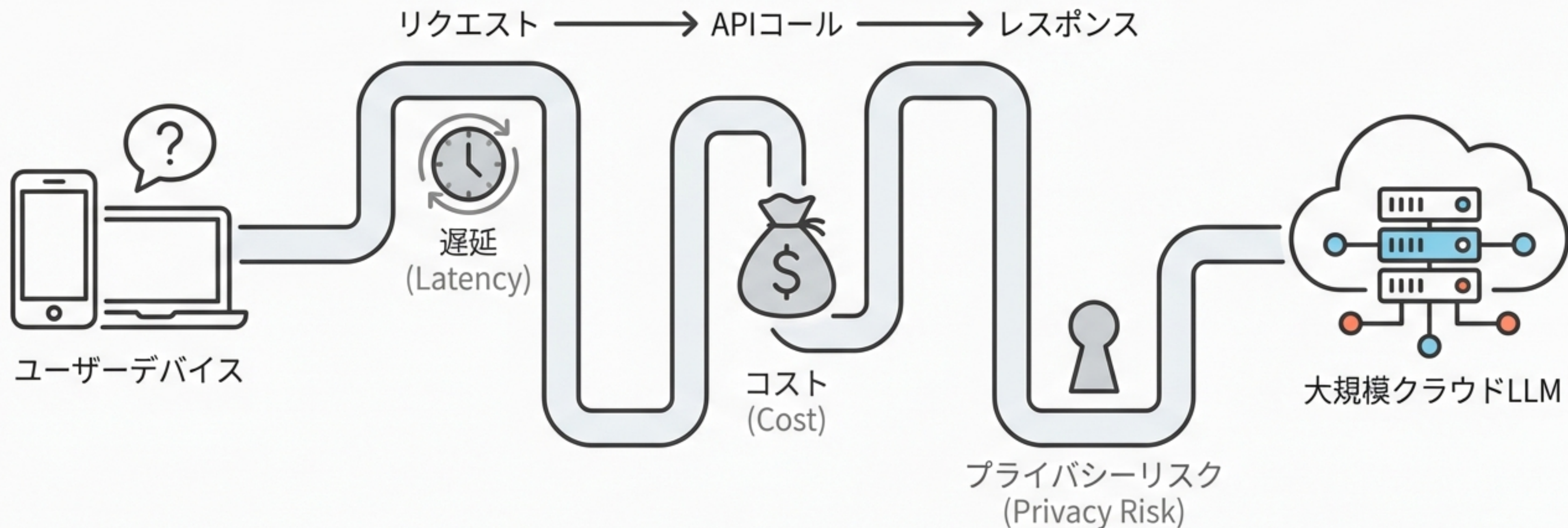
クラウドへの依存から脱却し、デバイス上で自律的に機能する「アクションエージェント」へ。  
FunctionGemmaが拓く未来。





PRIMARY\_FONT, HEADLINE\_FONT\_WEIGHT

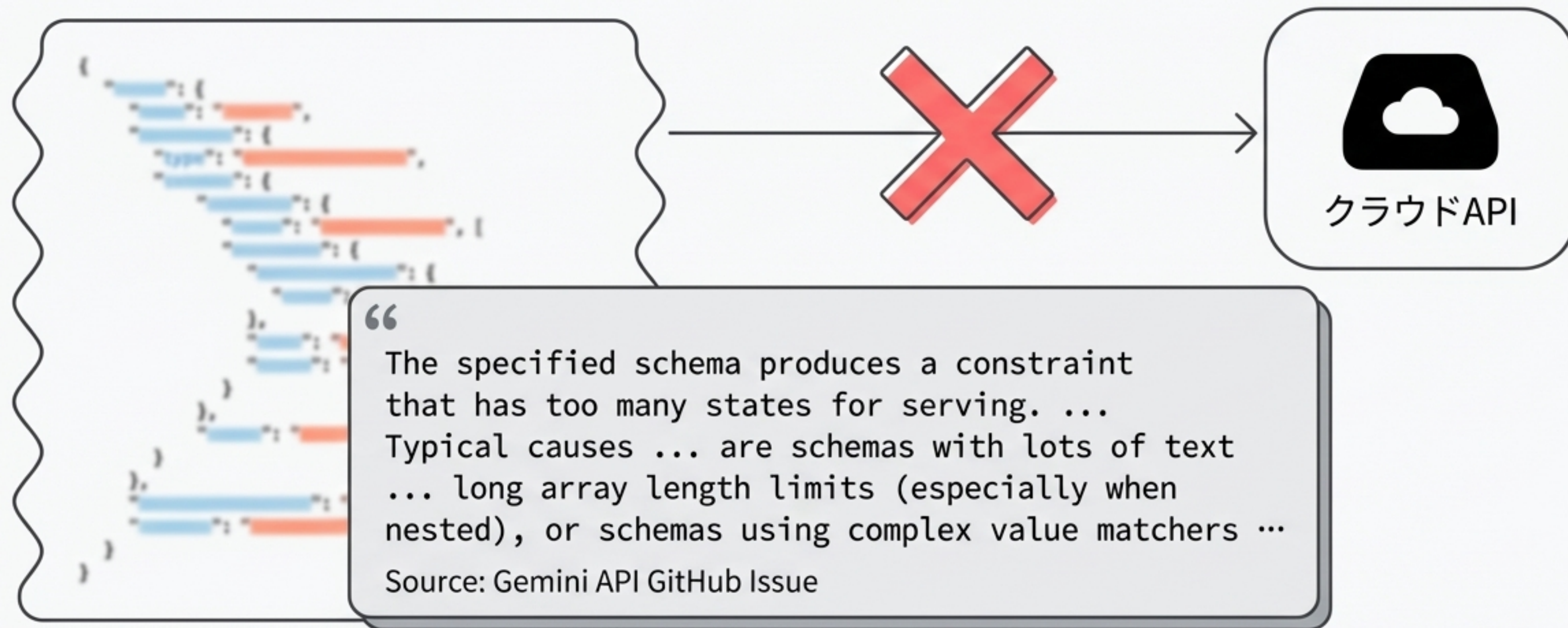
## 従来のクラウド中心アプローチが抱える課題



複雑なラウンドトリップが引き起こす、避けられないエンジニアリング上のボトルネック。



# 開発者を悩ませる「隠れた壁」： 複雑なJSONスキーマの問題



この問題は、特にエンタープライズ向けの複雑なツールやレガシーシステムと連携する際に、  
深刻なブロッカーとなりえます。

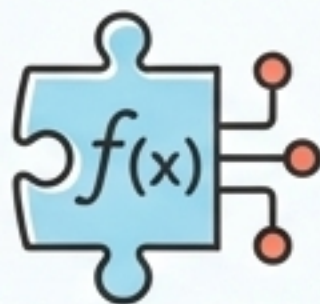


# 新時代の幕明けを告げる FunctionGemma

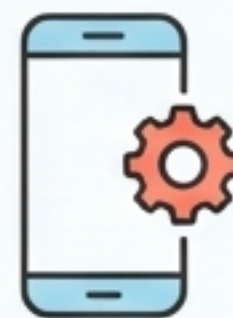
デバイス上で自律的に動作する「アクションエージェント」を実現するために生まれた、軽量・特化型モデル。



**軽量**  
(Lightweight)



**関数呼び出し特化**  
(Specialized for Function Calling)



**オンデバイス**  
(On-Device)



**オープン**  
(Open Weights)



# FunctionGemma 技術仕様



## アーキテクチャ

Gemma 3 270Mがベース



## パラメータ数

270M（約2.7億）



## 用途

テキストベースの関数呼び出し（ツール使用）に特化



## ライセンス

責任ある商用利用が可能なオープンウェイト



## コンテキスト長

32,768 トークン



## 語彙数

256,000（JSONの効率的なトークン化に貢献）

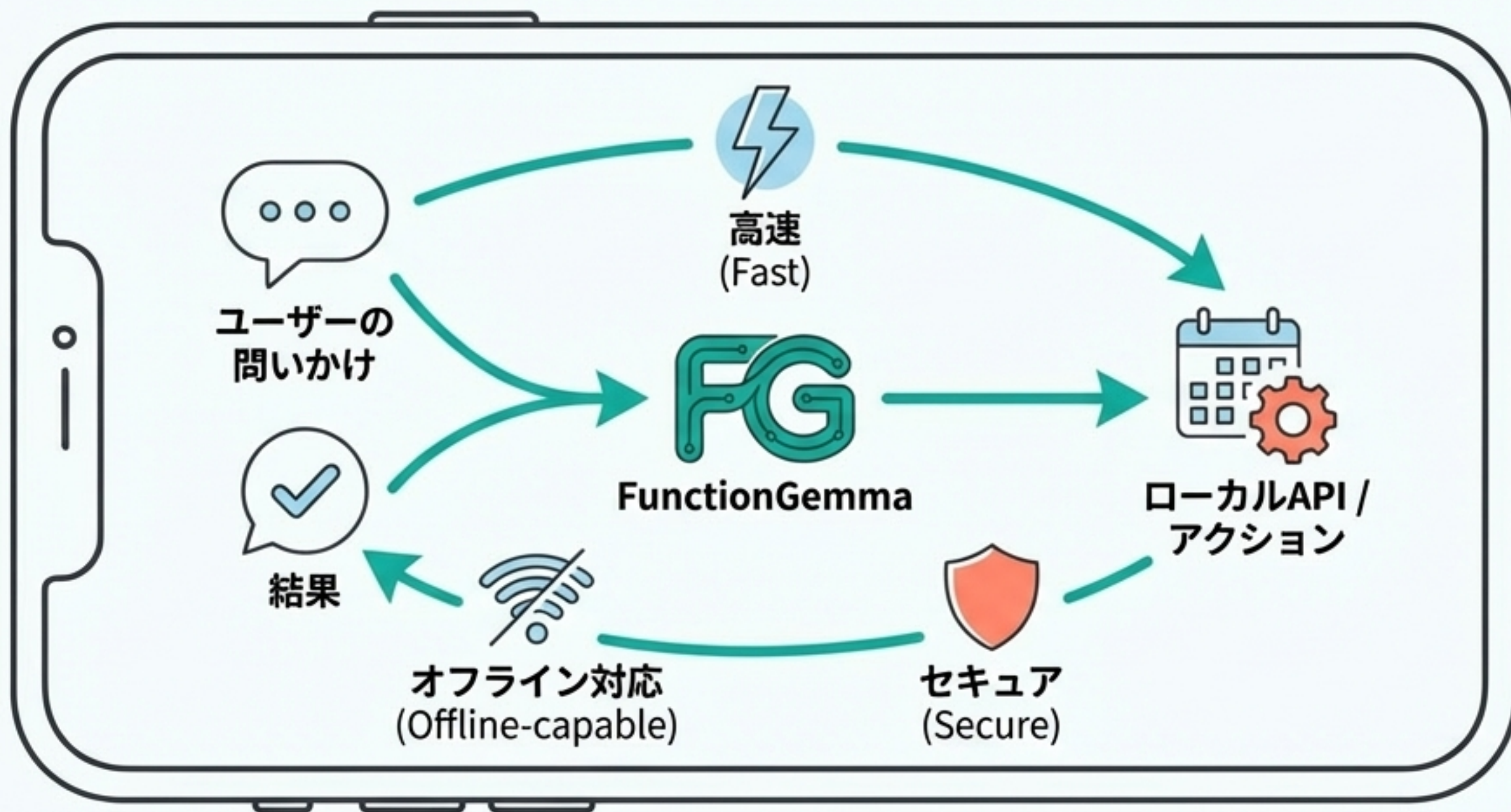


## 特徴

ファインチューニングによる特化を前提とした設計



# FunctionGemmaが実現する新しいワークフロー



クラウドへのラウンドトリップはもう不要。  
デバイス内で完結する、真のリアルタイム処理へ。



# なぜこれが開発者にとって重要なのか？



## プライバシー (Privacy)

ユーザーデータがデバイスから外部に出ない。  
機密性の高い情報を扱うアプリケーションに最適。



## スピード (Speed)

ネットワーク遅延がゼロ。ユーザー操作に対する  
瞬時のフィードバックを実現。



## コスト (Cost)

クラウドLLMのAPIコールが不要になり、運用コストを  
大幅に削減。単純なタスクのルーティングで効果を発揮  
（「ミドルウェア税」の廃止）。



## 信頼性 (Reliability)

オフラインでも動作。インターネット接続に依存しない  
安定した機能を提供可能。



# 優れたベースモデルから、本番環境で使える特化モデルへ

「FunctionGemmaは、ファインチューニングされることを前提に設計されています。」



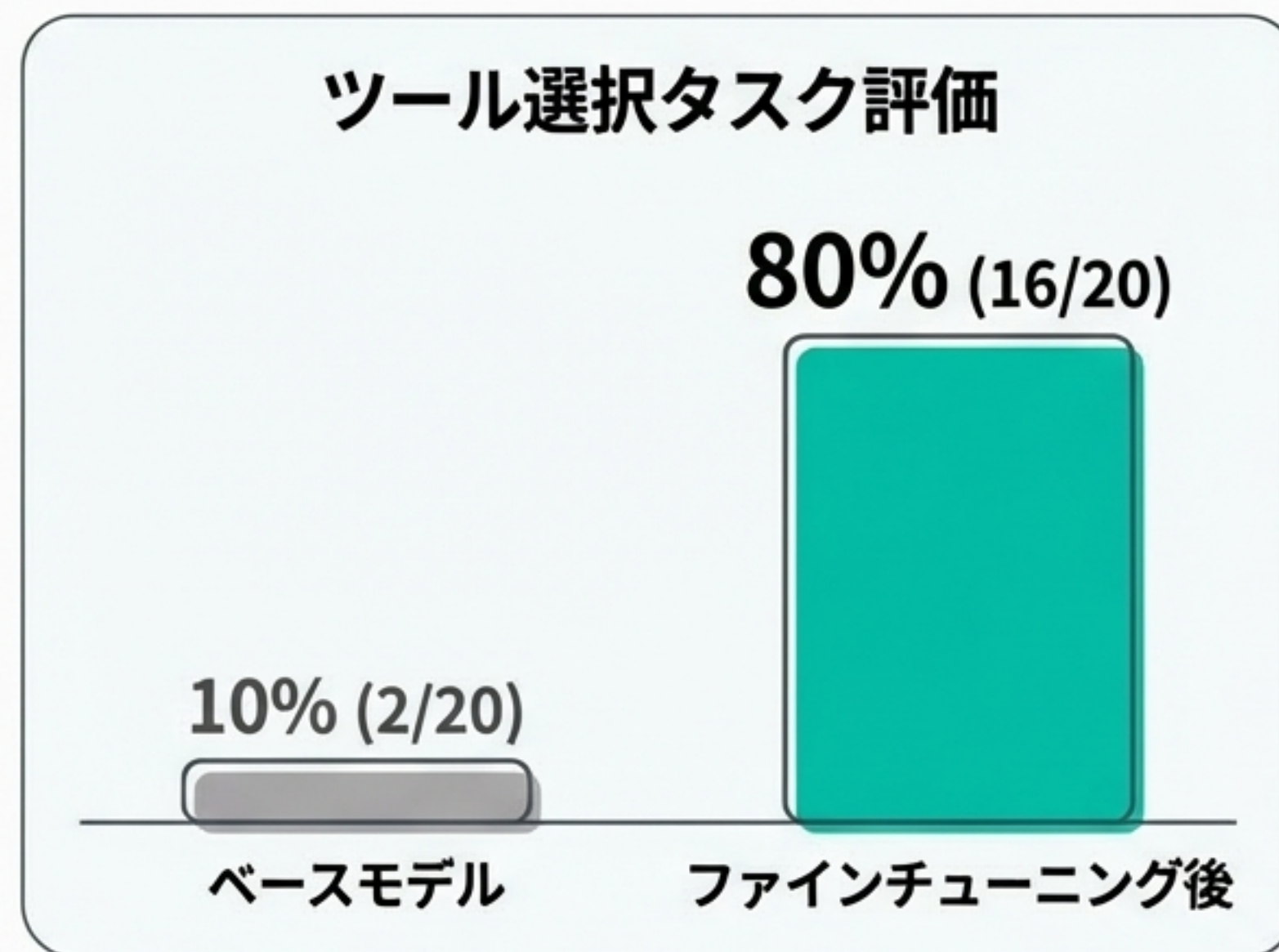
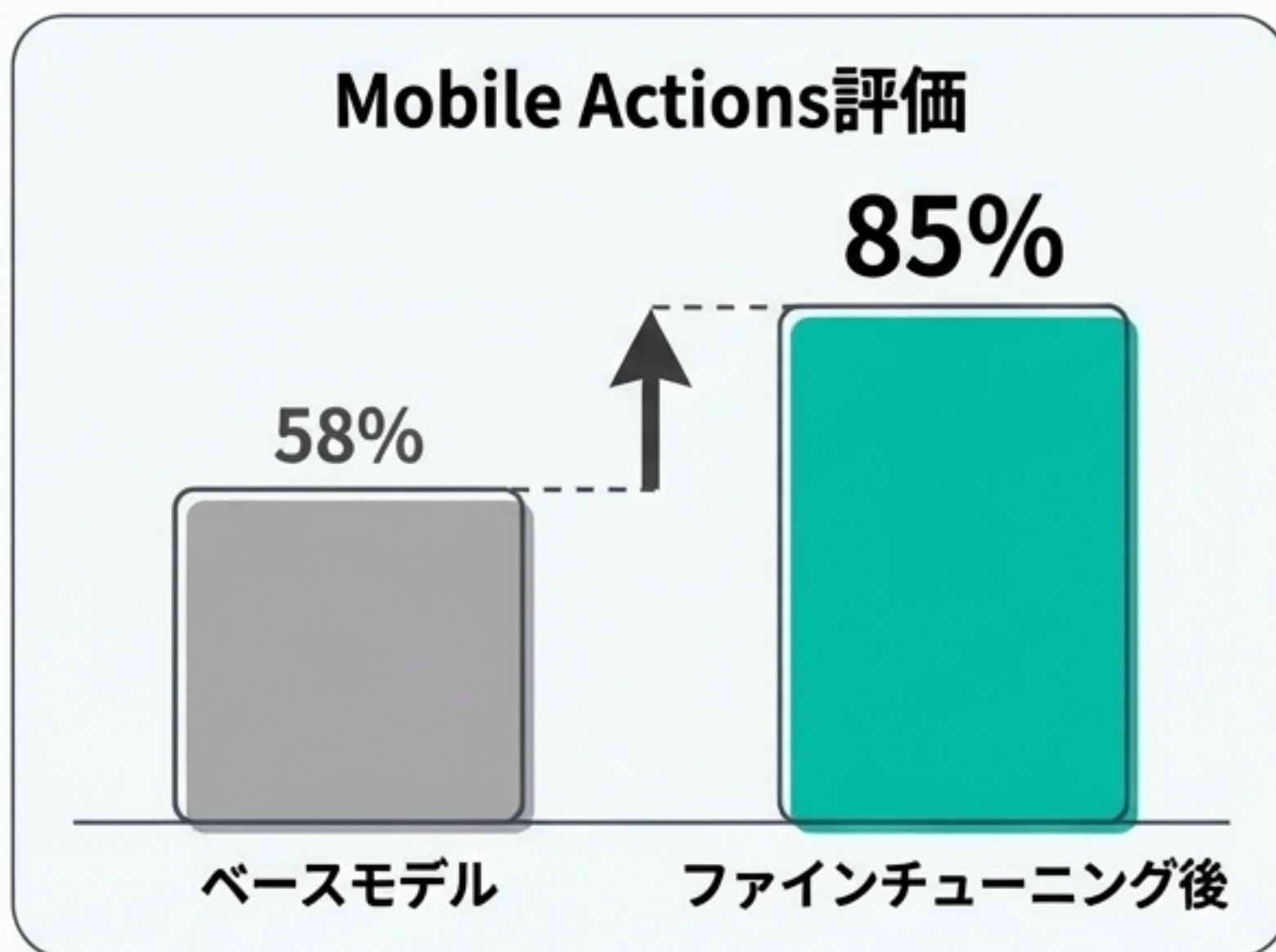
ベースモデル



特化モデル



## ファインチューニングの効果は、数字が証明する



わずかなトレーニングで、モデルの挙動は劇的に改善され、特定のタスクにおいて実用的な信頼性を獲得します。



# ファインチューニングが解決する4つの主要な課題



## 1. モデル蒸留 (Model Distillation)

大規模モデルの知識を小規模な FunctionGemma に転移させ、特定のワークフローを効率的に再現する。



## 2. 非標準スキーマへの対応 (Handling Non-Standard Schemas)

レガシーシステムや独自フォーマットなど、一般的でない複雑なデータ構造をモデルに学習させる。



## 3. コンテキストの最適化 (Optimizing Context Usage)

ツールの定義をモデルの重みに「焼き付ける」ことでプロンプトのトークン数を節約し、より長い対話にコンテキストを割り当てる。



## 4. 選択の曖昧性解消 (Resolving Selection Ambiguity)

複数のツール候補がある場合、企業ポリシーに基づき特定のツールを優先させるようにバイアスをかける。



# 実践例：社内情報か、Web検索か？

What is our Slack message retention policy?

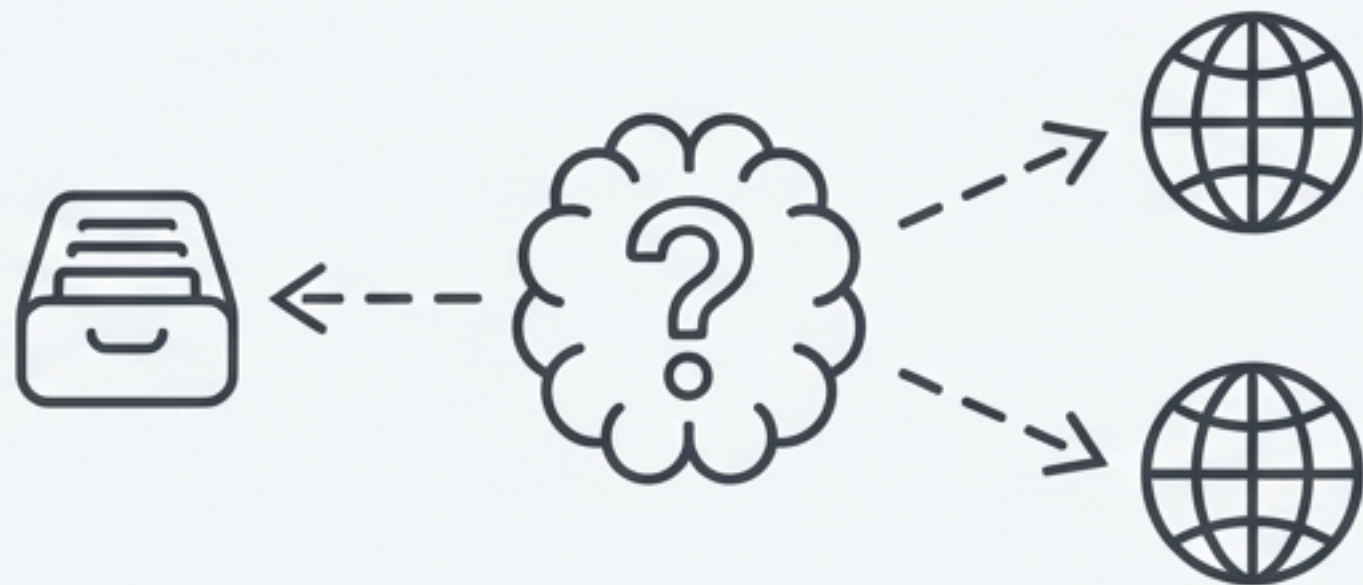


`search\_knowledge\_base` (社内ドキュメント検索)

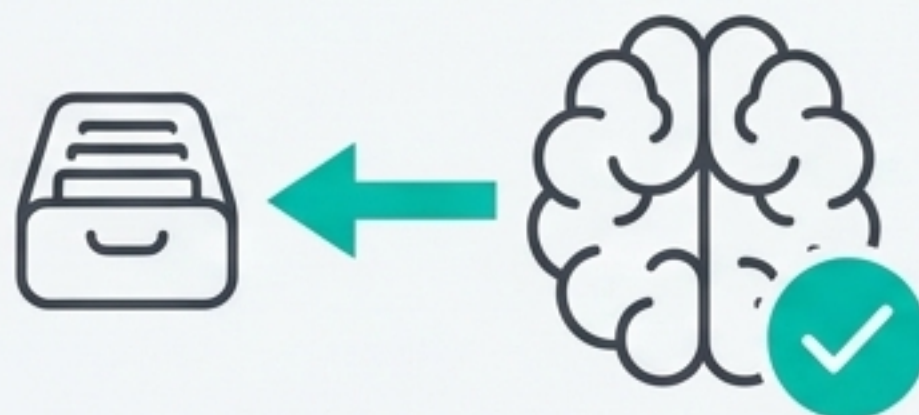


`search\_google` (公開Web検索)

## ベースモデルの思考



## ファインチューニング後のモデルの思考



ファインチューニングにより、モデルはクエリの意図を正確に解釈し、常に社内情報を優先するという『ポリシー』を学習します。



## ツール選択タスクにおける性能評価：Before & After

| Before Fine-tuning                    |                                                |                             |
|---------------------------------------|------------------------------------------------|-----------------------------|
| 成功率: 10% (2/20)                       |                                                |                             |
| Prompt                                | Output                                         | Result                      |
| Where can I find the floor plan...?   | <call:search_kb...><br><call:search_google...> | ⚠️ 両方を呼び出し<br>(Calls Both)  |
| What is the syntax for Python...?     | I cannot assist...                             | ❌ 呼び出し失敗<br>(Fails to Call) |
| What are the latest release notes...? | <call:search_kb...>                            | ❌ 間違ったツール<br>(Wrong Tool)   |

| After Fine-tuning                     |                         |                     |
|---------------------------------------|-------------------------|---------------------|
| 成功率: 80% (16/20)                      |                         |                     |
| Prompt                                | Output                  | Result              |
| Where can I find the floor plan...?   | <call:search_kb...>     | ✅ 正解!<br>(Correct!) |
| What is the syntax for Python...?     | <call:search_google...> | ✅ 正解!<br>(Correct!) |
| What are the latest release notes...? | <call:search_google...> | ✅ 正解!<br>(Correct!) |

ベースモデルは頻繁に混乱し不適切な応答を返しますが、ファインチューニング後は、意図した通りの.... ツール選択を高い精度で実行できるようになります。



# パフォーマンスの客観的評価指標： Berkeley Function-Calling Leaderboard (BFCL)

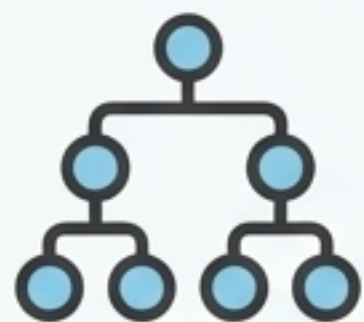


Berkeley Function-Calling Leaderboard

- 関数呼び出し（ツール使用）能力を評価するために設計された、**初の大規模・総合的ベンチマーク**。
- 様々なプログラミング言語に対応 (Python, Java, JS, SQL, REST API)。
- 単純な呼び出しだけでなく、並列・複数呼び出しも評価。
- さらに「適切なツールがない場合に呼び出しを控える」能力も評価。

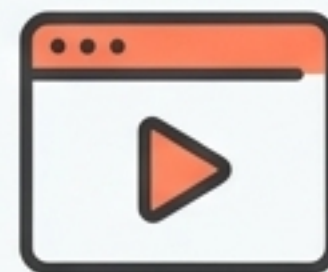


# BFCLはどのように性能を測定するのか？



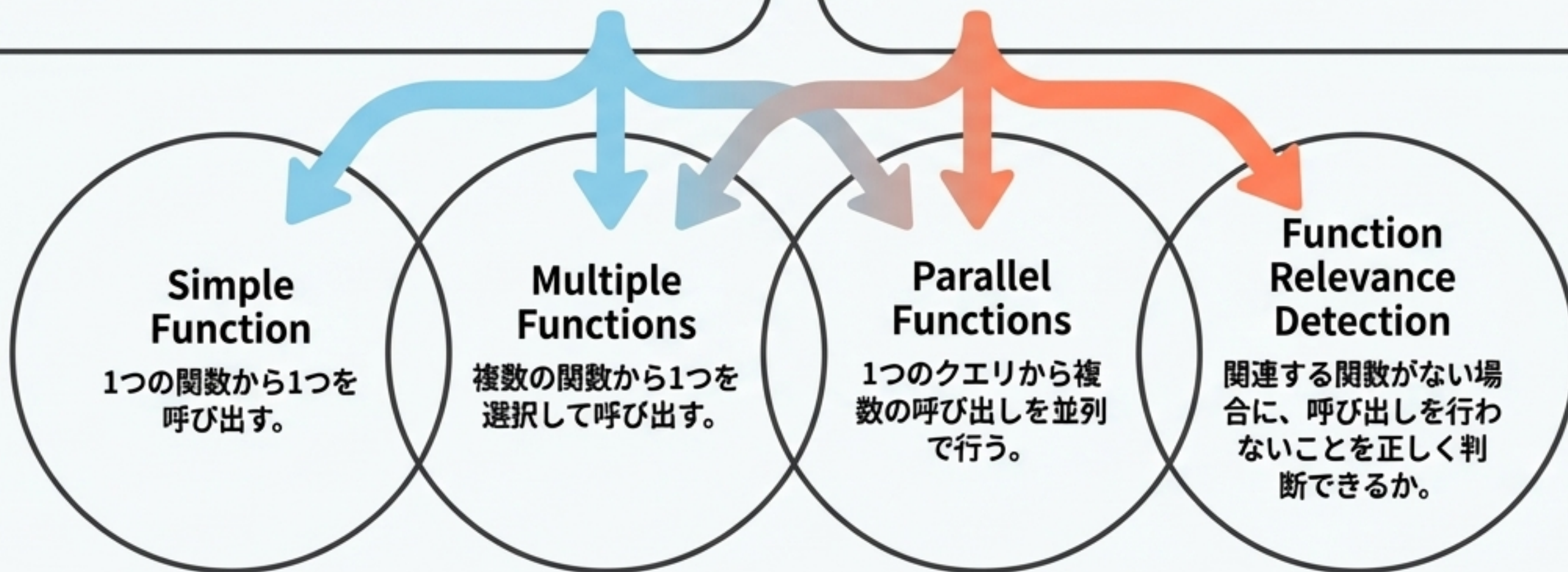
## AST評価 (Abstract Syntax Tree)

生成されたコードを構文木レベルで解析し、関数名、引数、型が正しいかを厳密に比較。実行環境がなくても評価可能。



## 実行可能評価 (Executable)

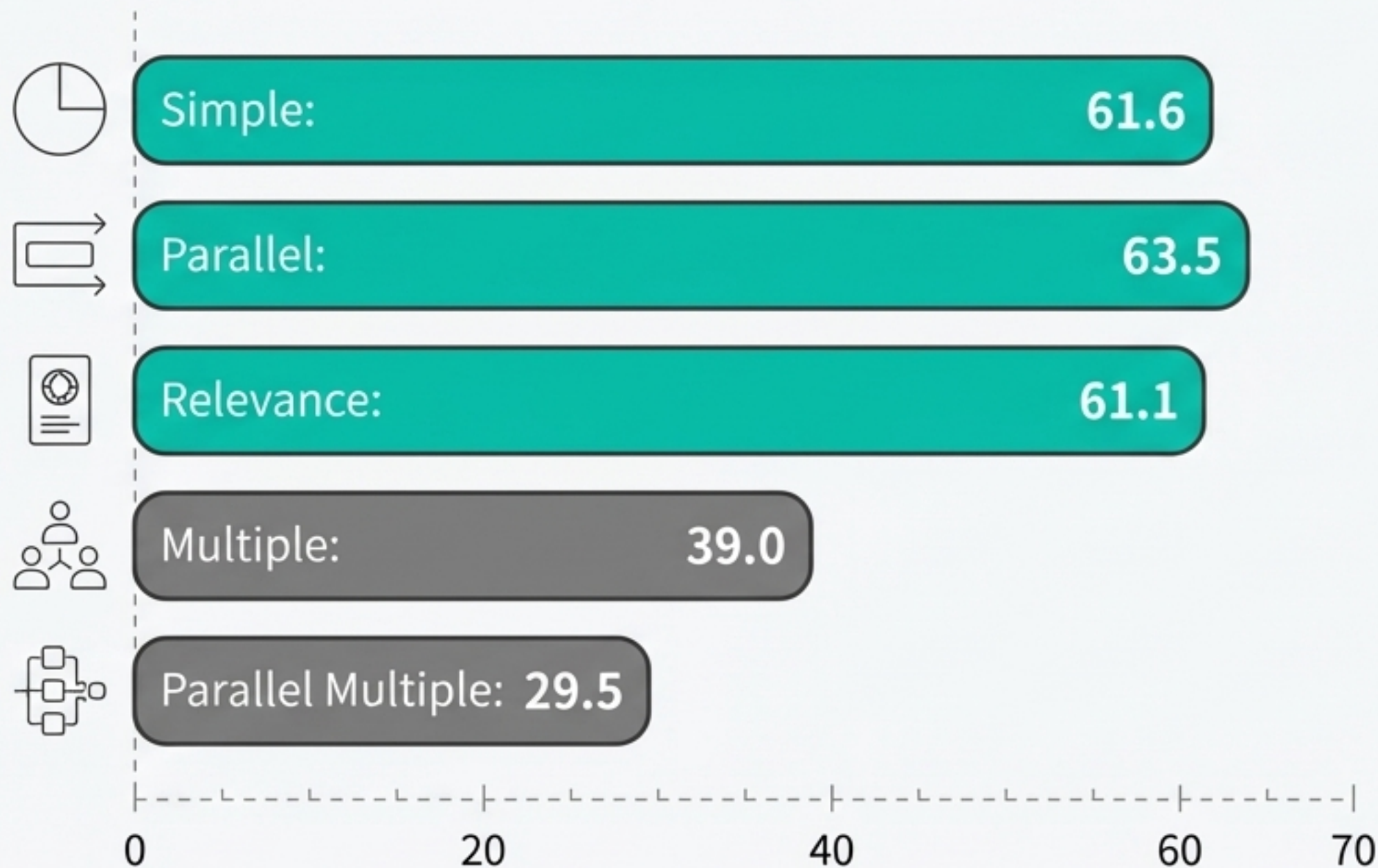
実際に生成されたコードを実行し、その出力が期待通りかを検証。より実用的な能力を測定。





# BFCLにおけるFunctionGemmaのスコア

FunctionGemma (270m) 0-shot Scores



基本的な単一・並列呼び出しでは堅牢な性能を発揮。

複数の選択肢から選ぶような、より複雑なシナリオでは、ファインチューニングによる性能向上の余地が大きいことを示唆。



# その性能を、あなたの手の中に：オンデバイス実行速度

~125

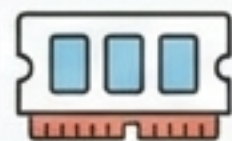
tokens/sec

デコード速度 (Decode)



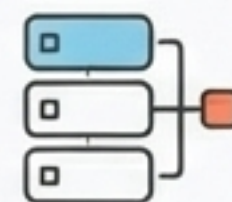
~550 MB

ピークメモリ使用量  
(Peak Memory)



0.3 秒

初回トークン生成時間  
(TTFT)



288 MB

モデルサイズ  
(Model Size)

測定条件: Samsung S25 Ultra (CPU), 量子化: dynamic\_int8

クラウドへの通信時間を遥かに凌駕する、圧倒的な応答速度。メモリ消費も少なく、  
モバイルデバイスでの常時稼働エージェントが現実のものに。



# FunctionGemmaを使った開発ワークフロー



## Step 1: Model's Turn (モデルの役割)

- 入力: ユーザーのプロンプトと利用可能なツールのリスト
- 出力: 呼び出すべき関数とその引数を含む、構造化されたオブジェクト



## Step 2: Developer's Turn (開発者の役割)

- 処理: モデルの出力を解析し、指定された関数を実際に実行する。
- 出力: 関数の実行結果



## Step 3: Final Response (最終応答)

- 入力: 関数の実行結果
- 処理: 実行結果を基に、ユーザーへの最終的な自然言語の応答を生成する。



## Code: Step 1 - モデルに関数呼び出しを生成させる

```
message = [  
    # ESSENTIAL SYSTEM PROMPT:  
    {"role": "developer", "content": "You are a model that can do function calling  
    with the following functions"},  
    {"role": "user", "content": "What's the weather in Tokyo?"},  
]  
  
# processor.apply_chat_template(...)
```

「`developer`」ロールのプロンプトが、モデルの関数呼び出しロジックを起動するスイッチの役割を果たします。

### 生成される出力例 (Generated Output Example):

```
<start_function_call>call:get_current_weather{location:<escape>Tokyo,  
Japan<escape>}<end_function_call>
```



## Code: Step 2 - 関数を実行し、結果をモデルに返す

```
# Parse the model's output string to get function name and args  
# ... (regex or other parsing logic) ...
```

```
# WARNING: This is a demonstration. ⚠
```

```
# In a real application, use a secure mapping.
```

```
function_name = calls[0]['name']  
arguments = calls[0]['arguments']  
result = get_current_weather(**arguments)
```

```
message.append({  
    "role": "tool",  
    "content": [{"name": function_name, "response": result}]  
})
```

モデルの出力をパースして安全に関数を実行し、その結果を決られたフォーマットで対話履歴追加することが、次のステップへの鍵となります。



# 広がるFunctionGemmaエコシステム

## プラットフォーム



Hugging Face

kaggle



Google Cloud  
Vertex AI



Ollama

## ファインチューニングツール



Unsloth



nVIDIA NeMo

## ライブラリ



Transformers



PyTorch



JAX



Keras

使い慣れたツールとプラットフォームで、すぐに開発を始められます。  
活発なコミュニティとエコシステムが、あなたの挑戦をサポートします。



# すぐに試せる：ファインチューニング・クックブック



Google公式レシピ

## Mobile Actions Fine-tuning

Androidのシステムツールを操作するエージェントを構築。公式のベストプラクティスを学ぶ。



Unslot提供ノートブック

## Reason before Tool Calling

ツールを呼び出す前に  
`<think>` タグで思考プロセスを生成させる、高度な推論能力のファインチューニング。



Unslot提供ノートブック

## Multi-Turn Tool Calling

複数の対話ターンにまたがる、より複雑なタスクを実行するエージェントの構築。

これらのノートブックを使えば、専門的なエージェントのファインチューニングを数分で始めることができます。



# チャットボットの先へ：真の「エージェント」を構築する

私たちは、単に質問に答えるAIから、現実世界で自律的にタスクを遂行するAIへと移行しています。



自律性  
(Autonomy)



プライバシー  
(Privacy)



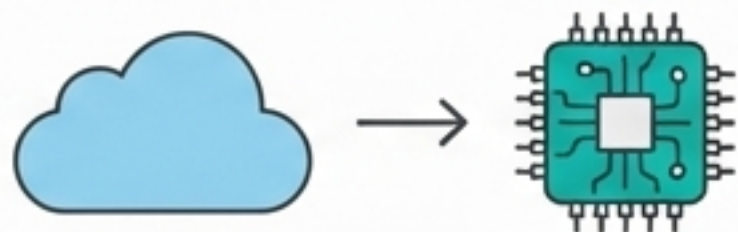
効率性  
(Efficiency)



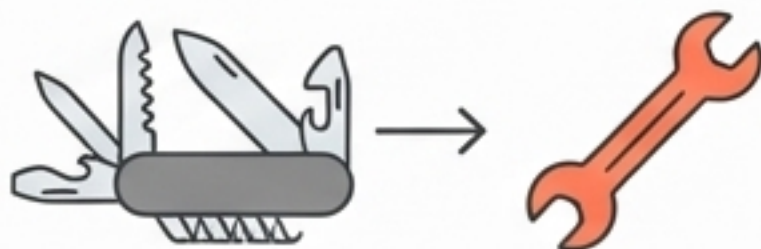
リアルタイム  
(Real-time)



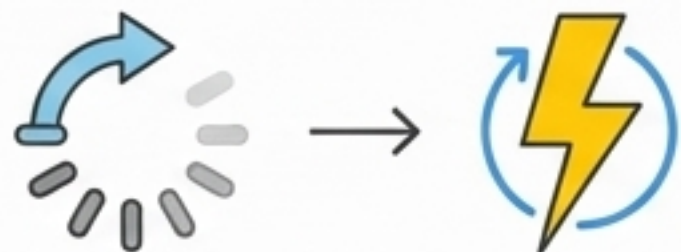
# What's New? それは、開発のパラダイムそのもの



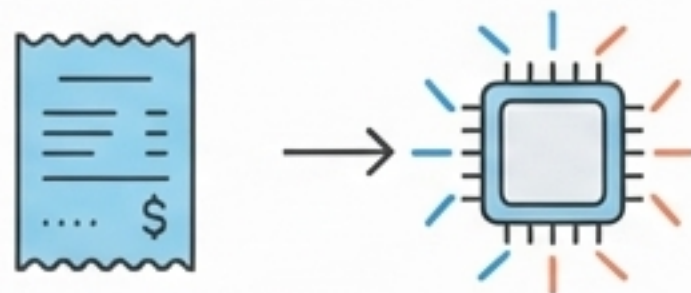
クラウドからエッジへ (From Cloud to Edge)



汎用モデルから特化モデルへ (From Generalist to Specialist)



遅延からリアルタイムへ (From Latency to Real-Time)



APIコストからローカル実行へ (From API Bills to Local Execution)

次世代のAIアプリケーションを構築するための、新しい常識が、今ここに。